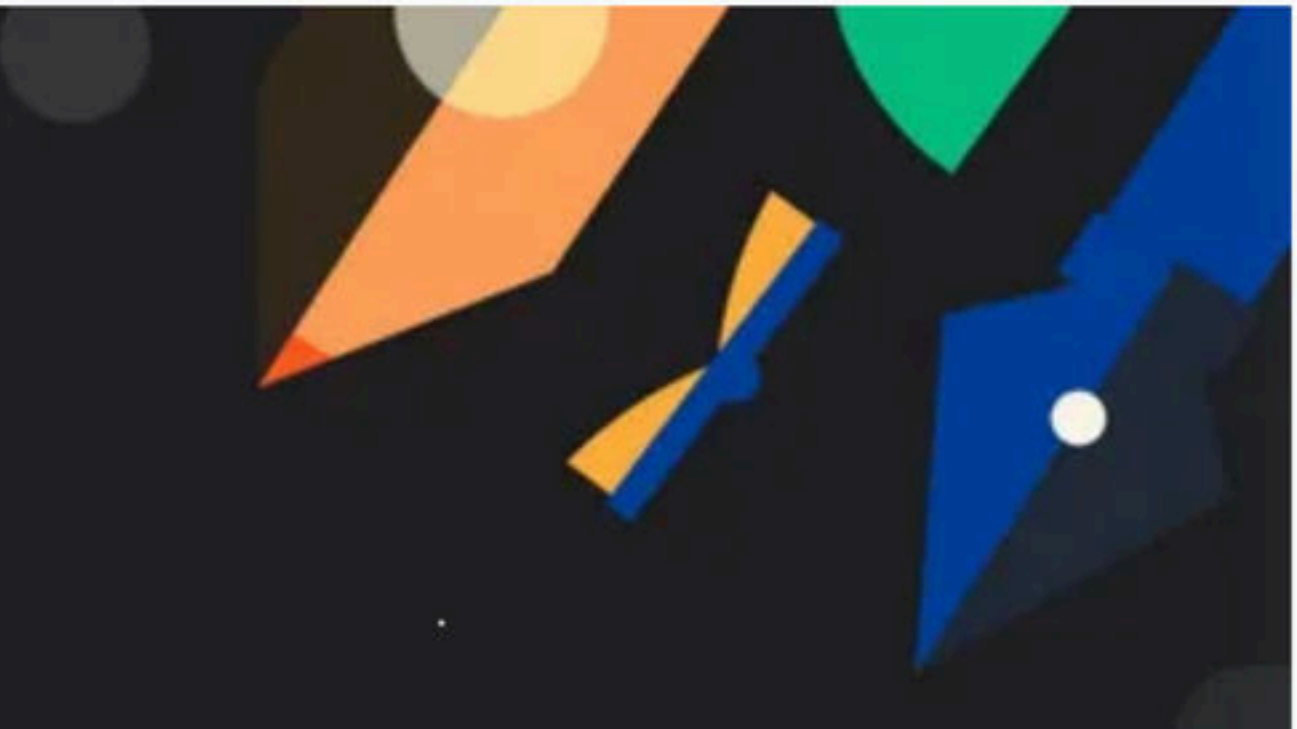




Doubts Class with Lakshay

Special class

Polymorphism in C++

1. Similar to Polymorphism in Life.
2. Polymorphism = Many Forms.
3. The ability of a single function or Operator to work in different ways based on the object it is acting upon or actual need.
4. A phenomenon that allows an object to have several different forms and behaviours.
5. Types
 1. Compile Time Polymorphism.
 2. Runtime Polymorphism.

Static Polymorphism

1. Aka, Compile Time Polymorphism.
2. Types
 1. Function Overloading
 2. Operator Overloading

Function Overloading

1. Overloading occurs when a class contains multiple methods sharing a name but differing in argument count or argument type.

```
class Add {  
public:  
    int sum(int x, int y) { return x + y; }  
    double sum(double x, double y) { return x + y; }  
};
```


Function Overloading

Operator Overloading

1. In C++, when operators are overloaded, they execute user-defined functions whenever used, allowing for customised behaviour.

```
class Complex {  
public:  
    int real, imag;  
    Complex(int r = 0, int i = 0) {real = r;    imag = i;}  
    Complex operator + (const Complex &obj) {  
        Complex res;  
        res.real = real + obj.real;  
        res.imag = imag + obj.imag;  
        return res;  
    }  
};
```

Operator Overloading

Operator Overloading

1. Below are the operators which can be overloaded in C++.

1. Arithmetic operators: `+`, `-`, `*`, `/`, `%`
2. Relational operators: `==`, `!=`, `<`, `>`, `<=`, `>=`
3. Logical operators: `&&`, `||`, `!`
4. Assignment operators: `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `<<=`, `>>=`, `&=`, `^=`, `|=`
5. Increment and decrement operators: `++`, `--`
6. Subscript operator: `[]`
7. Function call operator: `()`
8. Member access operators: `->`, `->*`
9. Allocation and deallocation: `new`, `new[]`, `delete`, `delete[]`
10. Bitwise operators: `&`, `|`, `^`, `~`, `<<`, `>>`
11. Other: `,`, `->*`, `()`, `[]`

Runtime Polymorphism

1. Function Overriding - makes function polymorphic.
2. Early vs Late binding.
3. Virtual Keyword - Way to achieve polymorphism by deferring binding decision to runtime.
4. Override keyword - Helps to make the intention clear and allows the compiler to enforce overriding rules, making your code safer and easier to understand.
5. Upcasting / Down-casting.

```
class Shape {
public:
    virtual void draw() {
        cout << "Drawing a generic shape" << endl;
    }
};

class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing a circle" << endl;
    }
};

class Rectangle : public Shape {
public:
    void draw() override {
        cout << "Drawing a rectangle" << endl;
    }
};
```

Runtime Polymorphism

Runtime Polymorphism

Final Keyword

1. In C++, the final specifier is used in two main contexts: with classes and with virtual member functions.
2. **Prevents Class Inheritance:** When you declare a class as final, it means that no other class can inherit from it.
3. **Preventing Virtual Function Overriding:** The final specifier can also be used with virtual functions to prevent them from being overridden in derived classes.

Polymorphism in Java

1. Similar to Polymorphism in Life.
2. Polymorphism = Many Forms.
3. The ability of a single function or Operator to work in different ways based on the object it is acting upon or actual need.
4. A phenomenon that allows an object to have several different forms and behaviours.
5. Types
 1. Compile Time Polymorphism.
 2. Runtime Polymorphism.

Static Polymorphism

1. Aka, Compile Time Polymorphism.
2. Types
 1. Method Overloading
 2. Operator Overloading

Method Overloading

1. Overloading occurs when a class contains multiple methods sharing a name but differing in argument count or argument type.

```
class Calculator {  
    // Method to add two integers  
    int add(int a, int b) {  
        return a + b;  
    }  
    // Overloaded method to add three integers  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

Method Overloading

Operator Overloading

1. Java does not support user-defined operator overloading but internally overloads certain operators (e.g., + for String concatenation and integer addition).

Runtime Polymorphism

1. **Function Overriding** - A Subclass can provide a specific implementation of a method that is already defined in its superclass.
2. **Dynamic Method Dispatch** [Upcasting] - A superclass reference variable can refer to a subclass object and dynamic dispatch is used to resolve the called method at runtime.

Runtime Polymorphism

Abstraction in C++

1. Delivering only essential information to the outer world while masking the background details.
2. It is a design and programming method that separates the interface from the implementation.
3. Real life e.g., various functionalities of AirPods but don't know the actual implementation/working
 1. To drive a car, one only needs to know the driving process and not the mechanics of the car engine

Abstraction in Header files

1. Function's implementation is hidden in header files.
2. We could use the same program without knowing its inside working.
3. E.g., Sort(), for example, is used to sort an array, a list, or a collection of items, and we know that if we give a container to sort, it will sort it, but we don't know which sorting algorithm it uses to sort that container.

Abstraction using Classes

1. Grouping data members and member functions into classes using access specifiers.
2. A class can choose which data members are visible to the outside world and which are hidden.

Abstraction in C++



```
class AbstractionExample{  
    private:  
        int num;  
        char ch;  
  
    public:  
        void setMyValues(int n, char c) {  
            num = n; ch = c;  
        }  
  
        void getMyValues() {  
            cout<<"Numbers is: "<<num<< endl;  
            cout<<"Char is: "<<ch<<endl;  
        }  
};
```

.....>

By making these data members private, we have hidden them from outside world.

.....>

These data members are not accessible outside the class.

⋮

The only way to set and get their values is through the public functions.

What is Abstract Class?

1. Class that contains at least one pure virtual function, and these classes cannot be instantiated.
2. It has come from the idea of Abstraction.

Design Strategy

1. Abstraction divides code into two categories: interface and implementation. So, when creating your component, keep the interface separate from the implementation so that if the underlying implementation changes, the interface stays the same.
2. In this instance, any program that uses these interfaces would remain unaffected and would require recompilation with the most recent implementation.

Abstraction in Java

1. Delivering only essential information to the outer world while masking the background details.
2. It is a design and programming method that separates the interface from the implementation.
3. Real life e.g., various functionalities of AirPods but don't know the actual implementation/working
 1. To drive a car, one only needs to know the driving process and not the mechanics of the car engine

Abstraction in Collections

1. E.g., Sort(), for example, is used to sort an array, a list, or a collection of items, and we know that if we give a container to sort, it will sort it, but we don't know which sorting algorithm it uses to sort that container.

```
import java.util.*;

public class SortExample {
    public static void main(String[] args) {
        List<Integer> numbers = new ArrayList<>();
        numbers.add(15);
        numbers.add(9);
        numbers.add(20);
        numbers.add(3);

        Collections.sort(numbers);

        System.out.println("Sorted Numbers:");
        for (int number : numbers) {
            System.out.println(number);
        }
    }
}
```

Abstraction using Classes

1. Grouping data members and member functions into classes using access specifiers.
2. A class can choose which data members are visible to the outside world and which are hidden.

What is Abstract Class?

1. An abstract class in Java is a class that cannot be instantiated on its own and is designed to be subclassed. It can serve as a superclass for other classes that share a common structure or behaviour.
2. An abstract class in Java delivers abstraction by serving as a template for its subclasses, allowing for the definition of a common interface without specifying the complete implementation details.
3. It has come from the idea of Abstraction.

Design Strategy

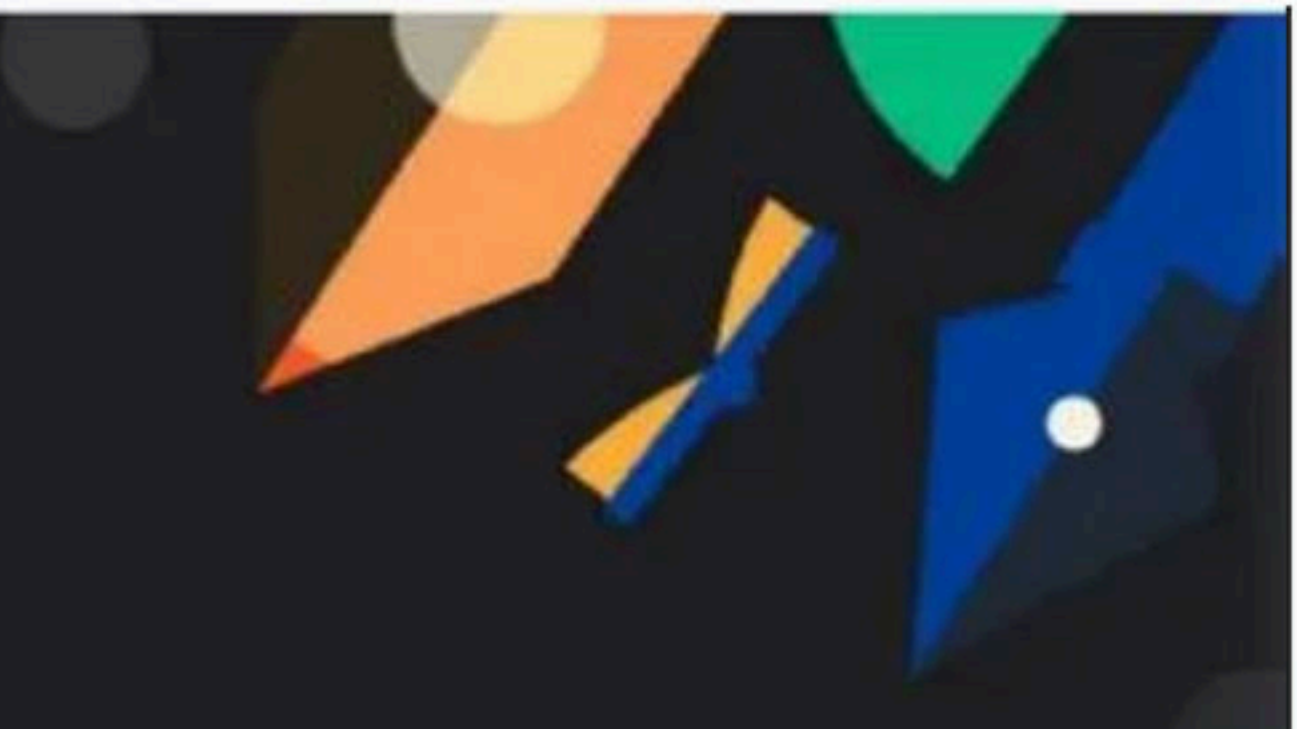
1. Abstraction divides code into two categories: interface and implementation. So, when creating your component, keep the interface separate from the implementation so that if the underlying implementation changes, the interface stays the same.
2. In this instance, any program that uses these interfaces would remain unaffected and would require recompilation with the most recent implementation.
3. Makes code modular and maintainable.

Abstract Class vs Interface

Feature	Abstract Class	Interface
<u>Instantiation</u>	Cannot be instantiated directly.	Cannot be instantiated.
Method Types	Can have both abstract and concrete methods.	Prior to Java 8, could only contain abstract methods. From Java 8 onwards, can also contain default and static methods with implementations.
Constructor	Can have constructors, which are called when a subclass is instantiated.	Cannot have constructors.
Access Modifiers	Methods can have any access modifiers (public, protected, private).	All methods are implicitly public. From Java 9, interfaces can also contain private methods but they must be used within the interface itself.
Inheritance	A class can extend only one abstract class due to single inheritance.	A class can implement multiple interfaces.
Fields and Constants	Can have fields and constants with any access modifier, and they can be non-static and non-final.	All fields are implicitly public, static, and final (constants).
Use Case	Ideal for classes that share a common base of properties and methods.	Ideal for defining a contract that different classes can implement, supporting multiple inheritance of type.

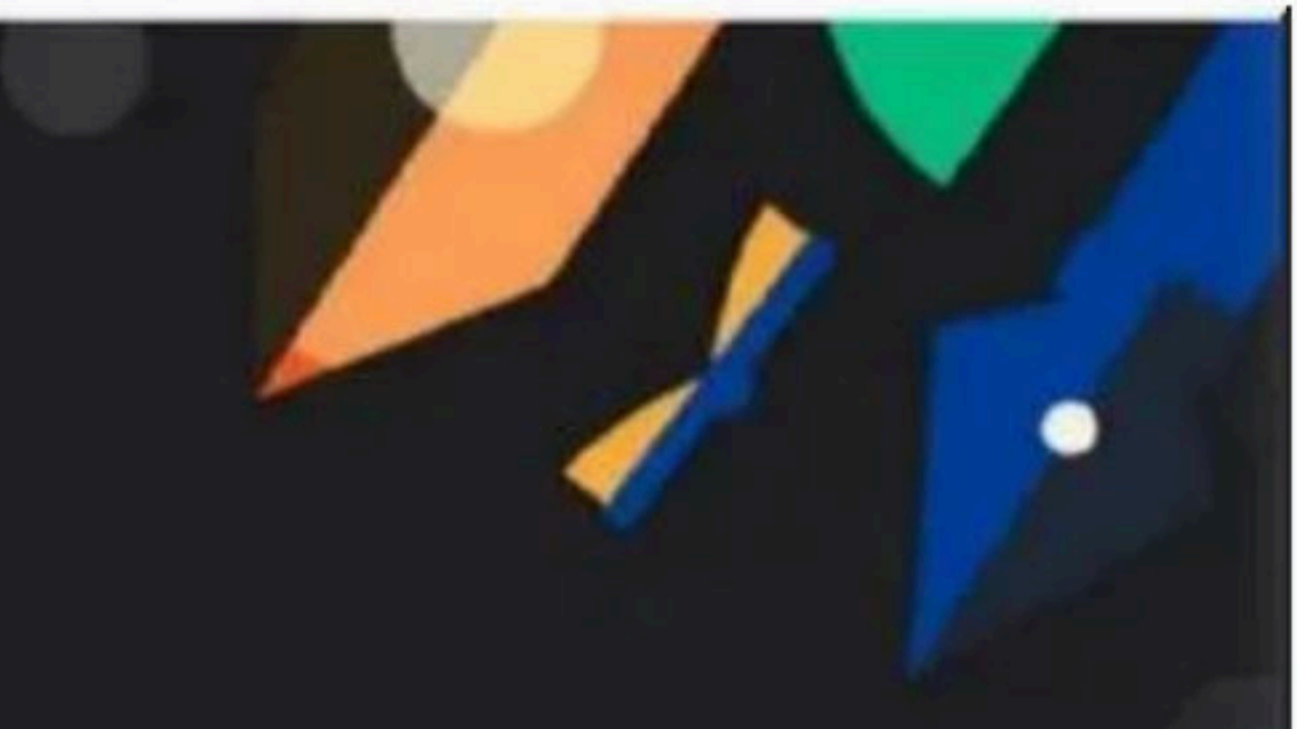
Advantages of Abstraction

1. Reduces Complexity
2. Increase Security
3. Increase Reusability
4. Avoid Code Duplication
5. Loose Coupling



SOLID Principles [Class-2]

Special class



Intro to SOLID Principles

Special class

SOLID Principles

- Lakshay

Good coder $\equiv$ Quality code

① Reusable code

② Extensible

③ Flexible

④ Stable $\rightarrow$ Exception Handling

⑤ Reliability

⑥ Modularity

⑦ Security -

⑧ Correctness



functional
= $\downarrow$
Tree
=

fun()



fun2()



3()

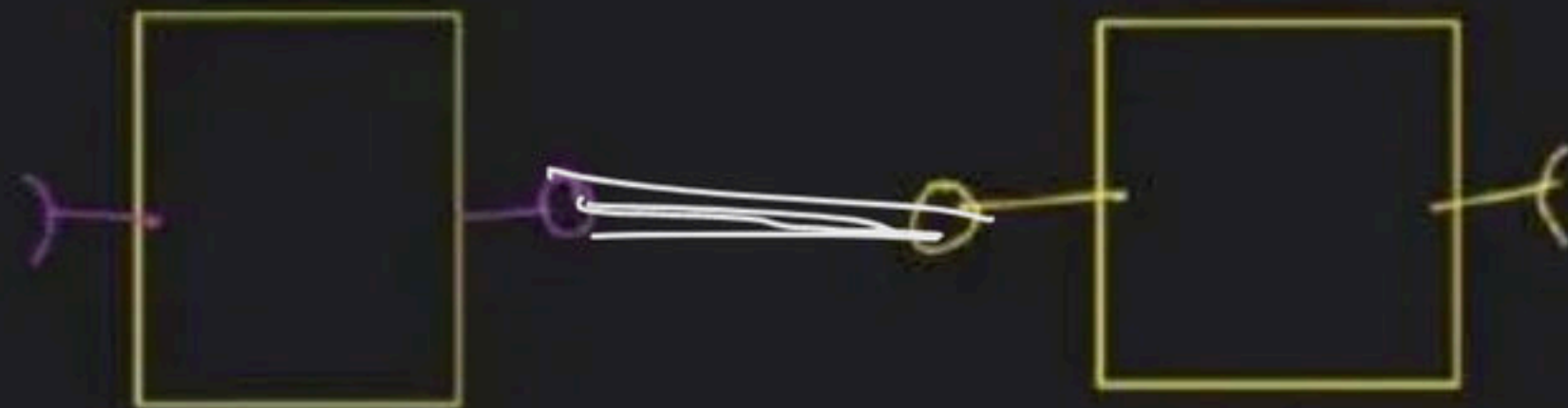
4()

...

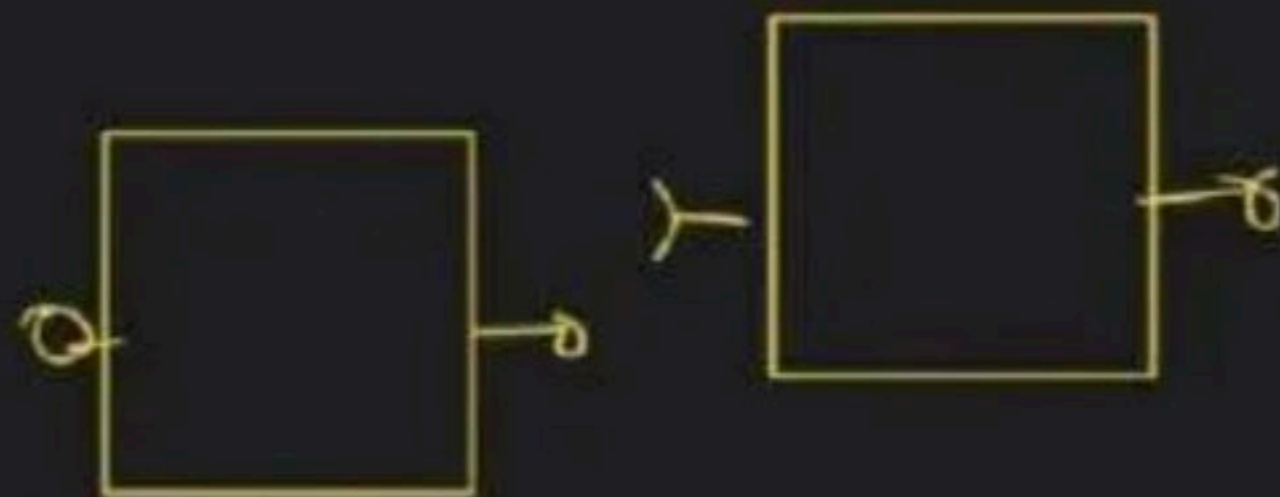
order
 $\downarrow$

OO (Leho)

Page 1

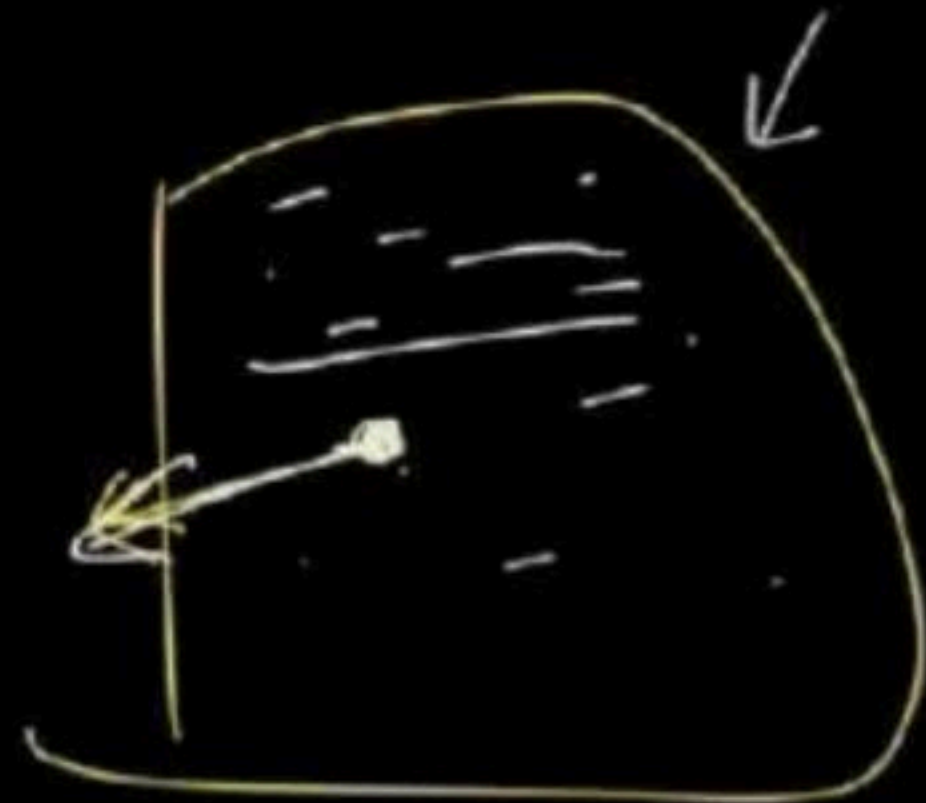


1



Purpose

1. Introduced by Robert Martin (Uncle Bob), named by Michael Feathers.
2. To make code more maintainable, easy to reuse.
3. To make it easier to quickly extend the system with new functionality without breaking the existing ones.
4. To make the code easier to read and understand, thus spend less time figuring out what it does and more time actually developing the solution. (Time Saving)



Single Responsibility Principle

1. A class should have one, and only one reason to change. This means that a class should only have one job or responsibility.
2. A class should only be responsible for one thing.
3. There's a place for everything and everything in its place.
4. Find one reason to change and take everything else out of the class.
5. **Importance**: Following SRP makes your code more modular, easier to understand, maintain, and extend. It helps in isolating functionalities, making debugging and testing more straightforward.

→)

Class Order {

void Pay ()

}

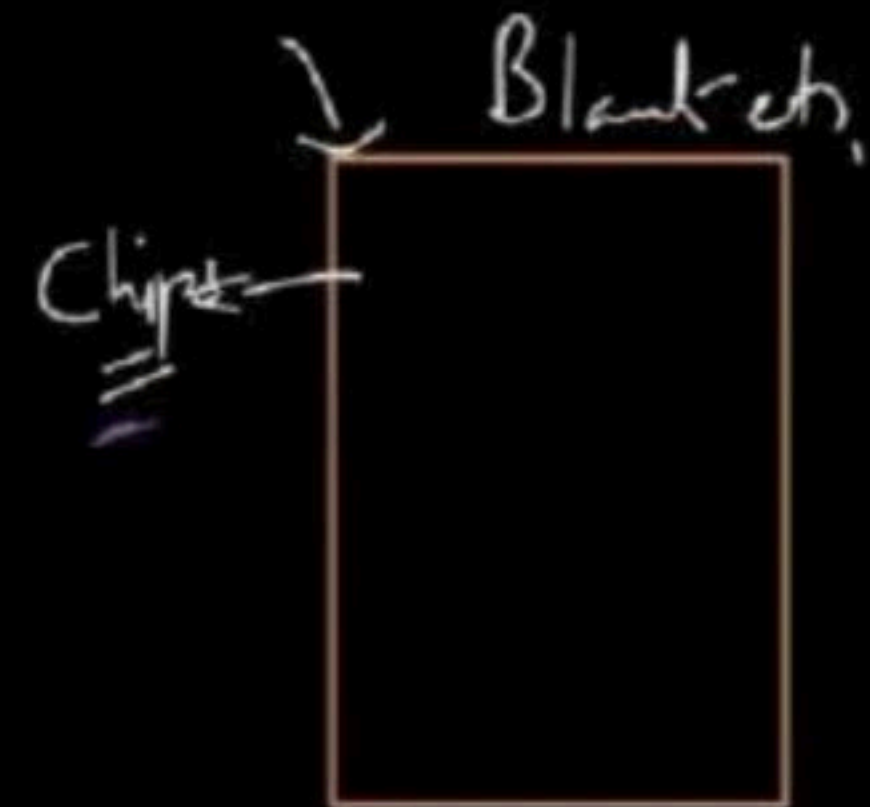
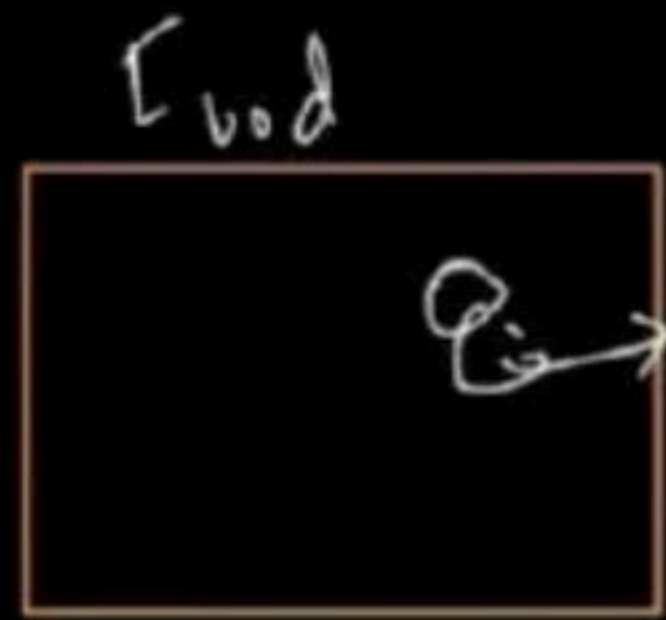
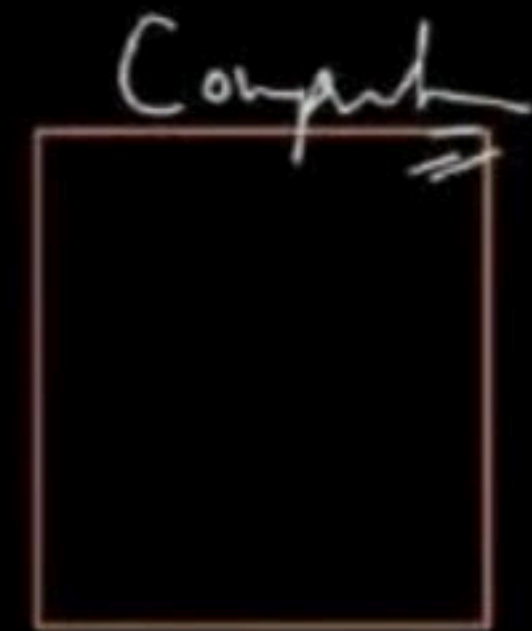
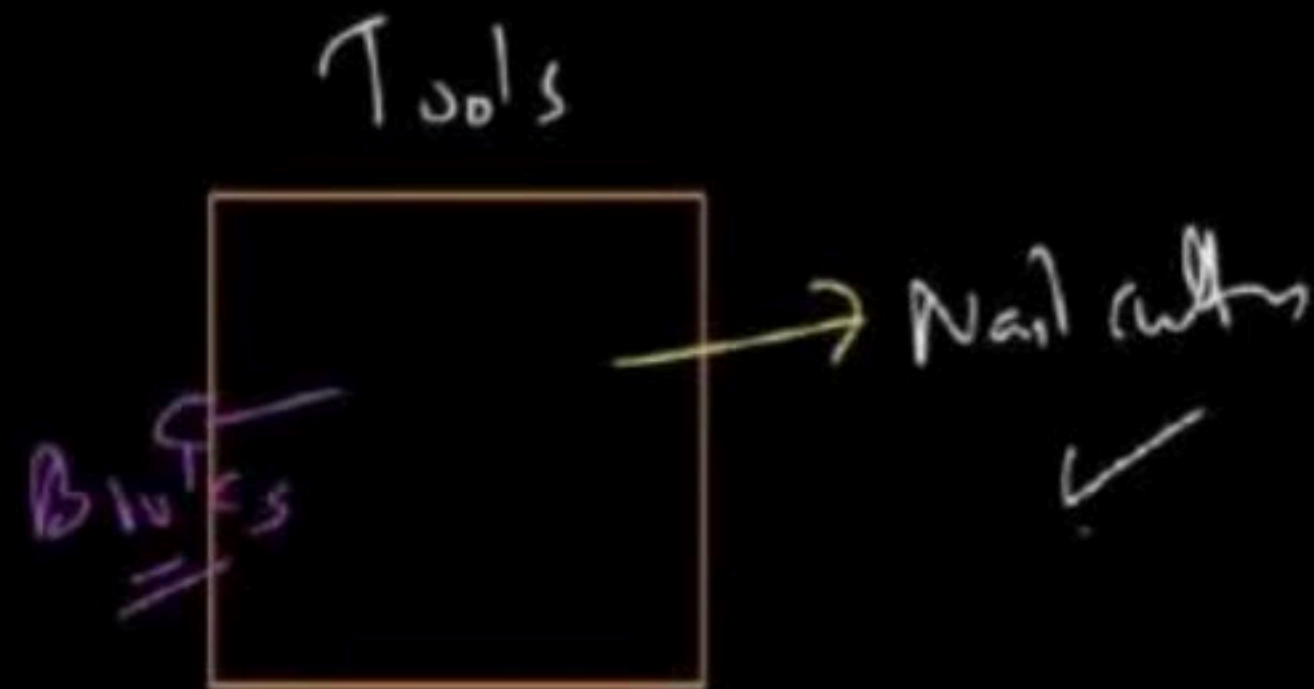
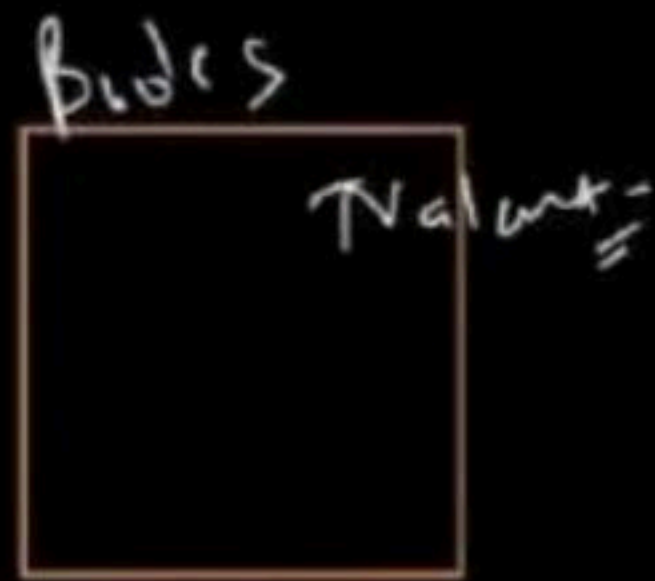
Order



Pay



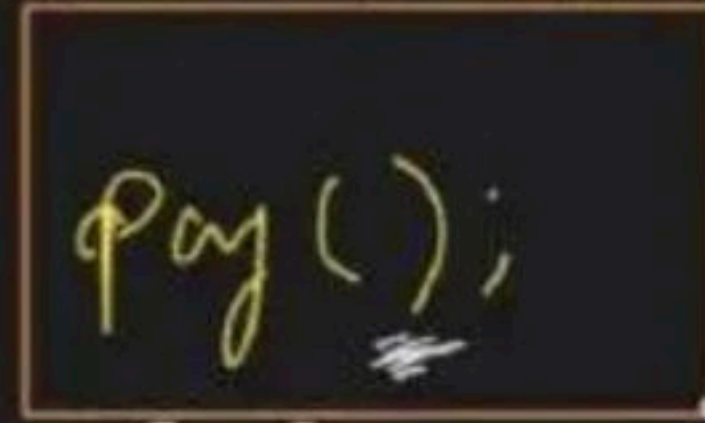
Single Responsibility Principle



Open-Closed Principle

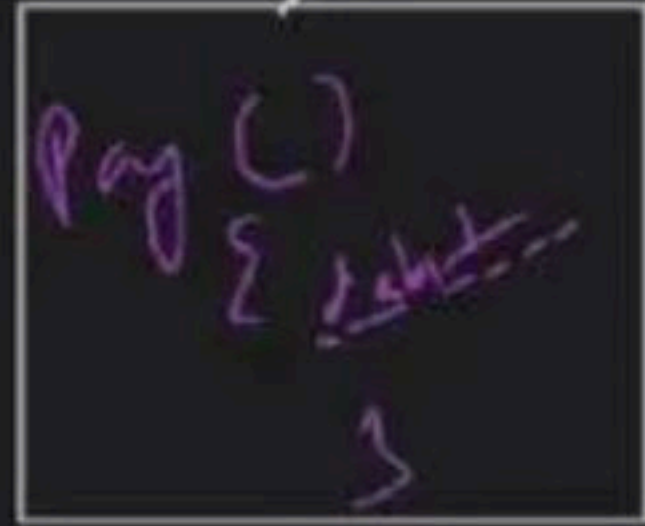
1. An entity should be open for extension but closed for modification. This means you should be able to add new functionality without changing the existing code.
2. Extend functionality by adding new code instead of changing existing code.
3. **Goal**: Get to a point where you can never break the core of your system.
4. **Importance**: OCP encourages a more stable and resilient codebase. It promotes the use of interfaces and abstract classes to allow for behaviors to be extended without modifying existing code.
5. Writing code structure in such a way new functionality can be added by adding new code not by modifying existing code.

Payment Processor

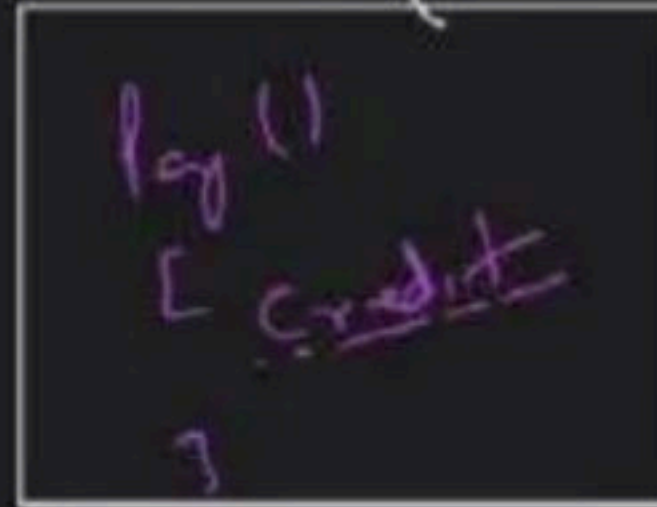


abstract class / interface

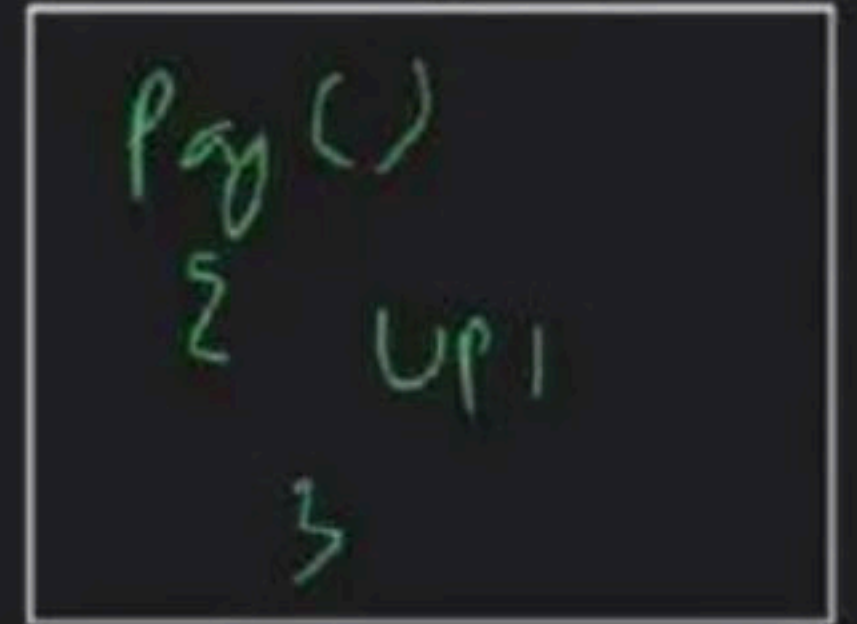
Debit Payment Processor



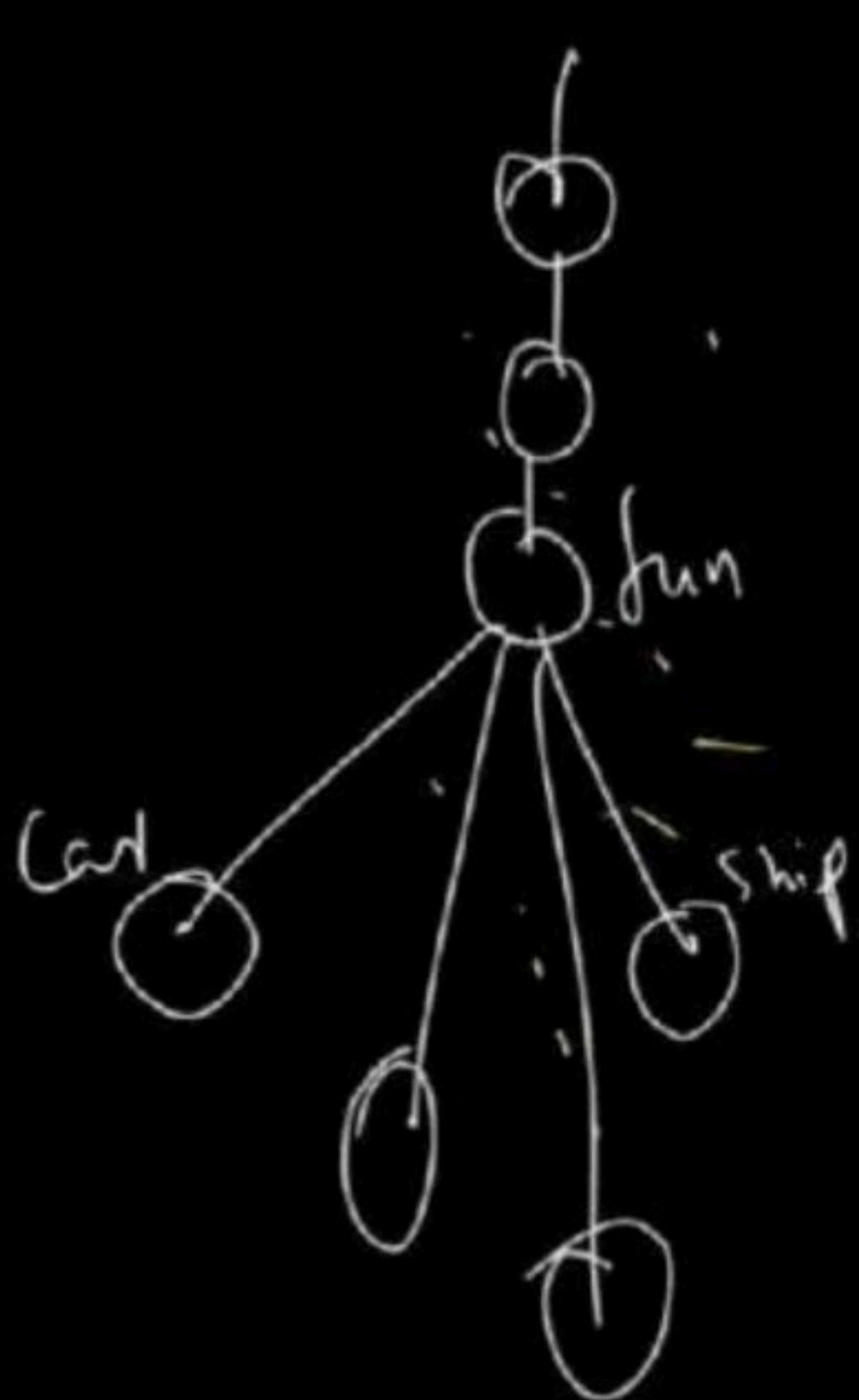
Credit Payment Processor



UPI Payment Processor



Open-Closed Principle



fun →

```
for(Vehicle vehicle : vehicles) {  
    switch(vehicle.getType()) {  
        case CAR:  
            vehicle.lock();  
            vehicle.go();  
            break;  
        case SHIP:  
            vehicle.balance();  
            vehicle.swim();  
            break;  
        case AIRPLANE:  
            vehicle.go();  
            vehicle.fly();  
            break;  
        case TANK:  
            vehicle.move();  
            vehicle.stop();  
            vehicle.fire();  
            break;  
    }  
    vehicle.stop();  
}
```

Bad

- ① if, switch
- ② Violate OCP
- ③ Cyclometric complexity
- ④ Downcasting
- ⑤ Typechecking
anti-ABS

Open-Closed Principle

```
1  do(Car v){
2      vehicle.lock();
3      vehicle.go();
4  }
5  do(Ship v){
6      vehicle.balance();
7      vehicle.swim();
8  }
9  do(Airplane v){
10     vehicle.go();
11     vehicle.fly();
12 }
13 do(Tank v){
14     vehicle.move();
15     vehicle.stop();
16     vehicle.fire();
17 }
18
19 execute(List<Vehicle> vehicles){
20     for(Vehicle vehicle : vehicles) {
21         do(vehicle);
22         vehicle.stop();
23     }
24 }
```

Compile X
↳ Time
Error

→ Down casting

Open-Closed Principle

```
1  do(Car vehicle){  
2      vehicle.lock();  
3      vehicle.go();  
4  }  
5  do(Ship vehicle){  
6      vehicle.balance();  
7      vehicle.swim();  
8  }  
9  do(Airplane vehicle){  
10     vehicle.go();  
11     vehicle.fly();  
12 }  
13 do(Tank vehicle){  
14     vehicle.move();  
15     vehicle.stop();  
16     vehicle.fire();  
17 }  
18  
19 execute(List<Vehicle> vehicles){  
20     for(Vehicle vehicle : vehicles) {  
21         if(vehicle instanceof Car){  
22             do((Car) vehicle)  
23         }  
24         if(vehicle instanceof Tank){  
25             do((Tank) vehicle)  
26         }  
27     }  
28 }
```

Downcasting

Open-Closed Principle

Vehicle

do();
stop();

Car
do!?!
stop~

```
1 interface Vehicle{
2     do();
3     stop();
4 }
5 class Car implements Vehicle{
6     do(){
7         lock();
8         go();
9     }
10    ...
11 }
12 class Ship implements Vehicle{
13     do(){
14         balance();
15         swim();
16     }
17     ...
18 }
19 class Airplane implements Vehicle{
20     do(){
21         go();
22         fly();
23     }
24     ...
25 }
26 class Tank implements Vehicle{
27     do(){
28         move();
29         stop();
30         fire();
31     }
32     ...
33 }
34
35 ArrayList<Vehicle> vehicles = new ArrayList<>();
36 for(Vehicle vehicle : vehicles) {
37     vehicle.do();
38     vehicle.stop();
39 }
```





```
1  interface Vehicle{
2      do();
3      stop();
4  }
5  class Car implements Vehicle{
6      do(){
7          lock();
8          go();
9      }
10     ...
11 }
12 class Ship implements Vehicle{
13     do(){
14         balance();
15         swim();
16     }
17     ...
18 }
19 class Airplane implements Vehicle{
20     do(){
21         go();
22         fly();
23     }
24     ...
25 }
26 class Tank implements Vehicle{
27     do(){
28         move();
29         stop();
30         fire();
31     }
32     ...
33 }
34
35 execute(List<Vehicle> vehicles){
36     for(Vehicle vehicle : vehicles) {
37         vehicle.do();
38         vehicle.stop();
```


Liskov Substitution Principle



1. Any derived class should be able to substitute its parent class without the consumer knowing it.
2. Every part of the code should get the expected result no matter what instance of a class you send to it, given it implements the same interface.
3. If a function takes a Base class as parameter then, this code should work for all the derived classes.
4. LSP insures that the good application i.e., built using abstraction does not break.
5. It states that the objects of a subclass should behave the same way as the objects of the superclass, such that they are replaceable.
6. Child class should be able to do what a parent class can.
7. **Goal:** The goal of LSP is to ensure that a subclass can stand in for its superclass. This principle helps in maintaining the correctness of the program when objects of a superclass are replaced with objects of a subclass.

Base class ABS

Bird {

fly()

eat()

}

{ fun (Bird b)

{

b.eat() =

if !instanceof(Ostrich)
b.fly() =

}

Sparrow {

eat()
fly()

}

Ostrich {

fly()

{
return "Ostrich
can't fly"

Exception

Error

eat()
→ fly()

}

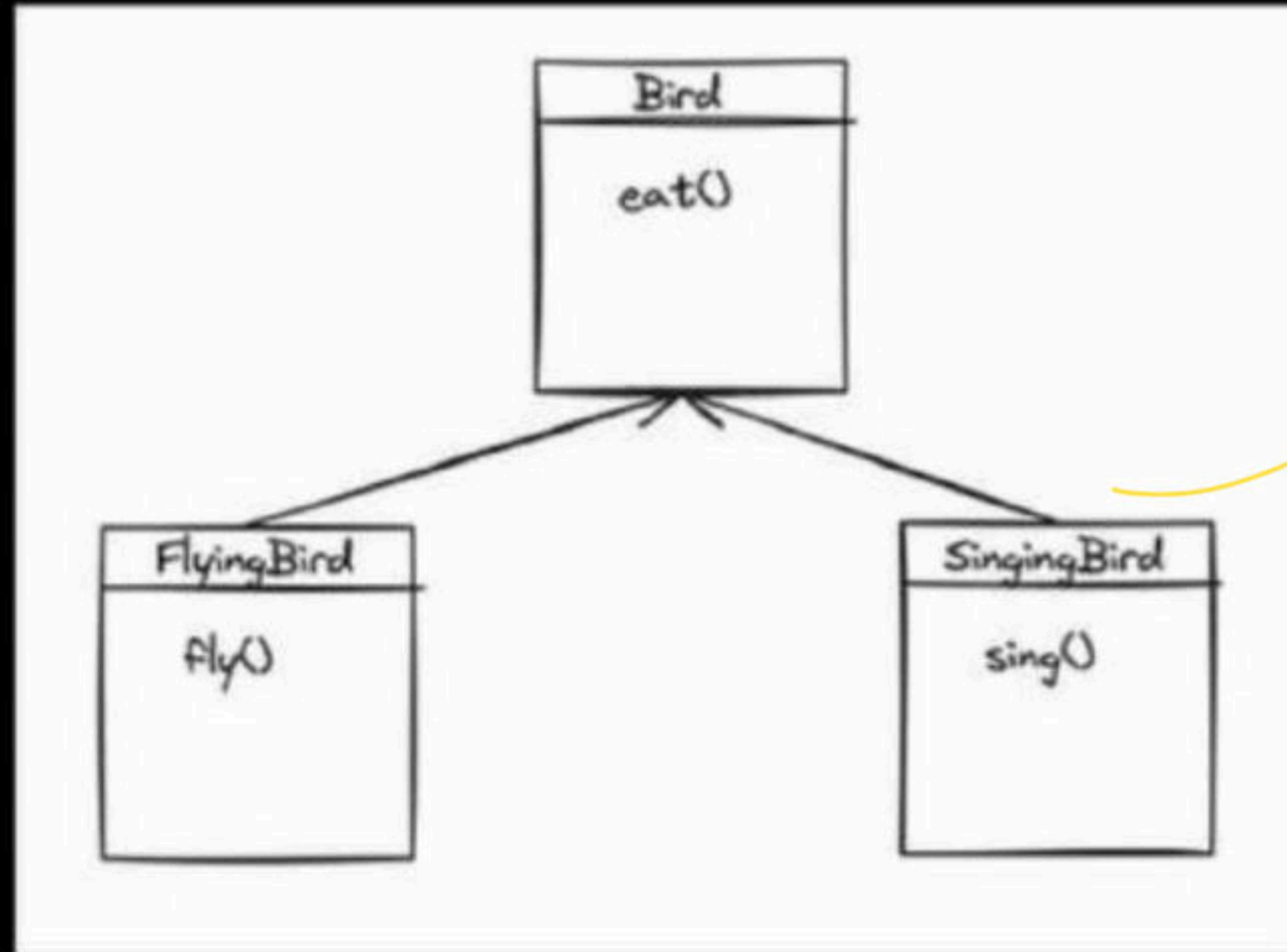
Liskov Substitution Principle

1. Bird Example



Liskov Substitution Principle

1. Bird Example




```
Bird {  
    eat();  
}
```

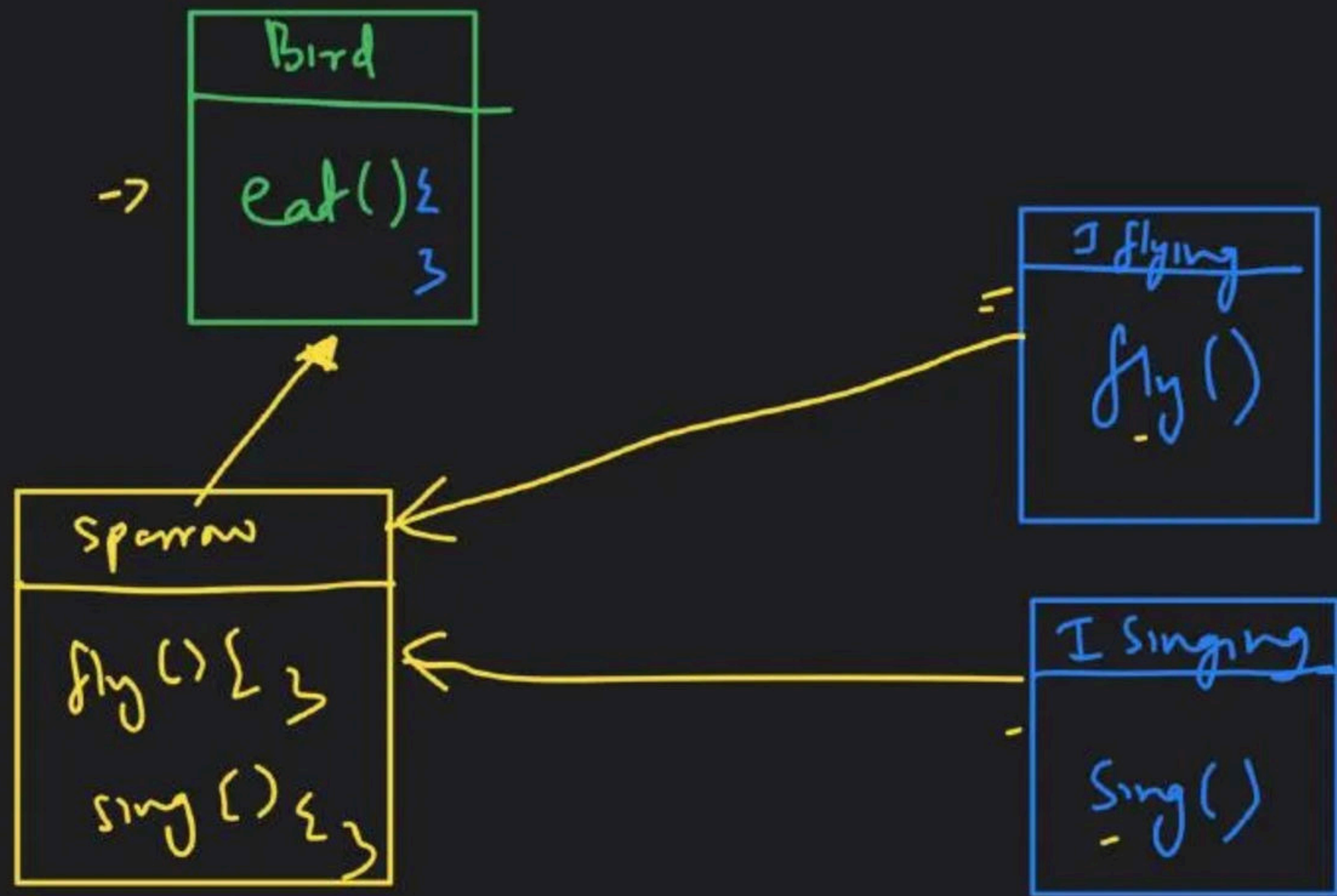
```
↓  
IFlyingBird {  
    fly();  
}
```

Sparrow extend Bird implement IFlyBr,

```
↓  
ISingBird {  
}
```

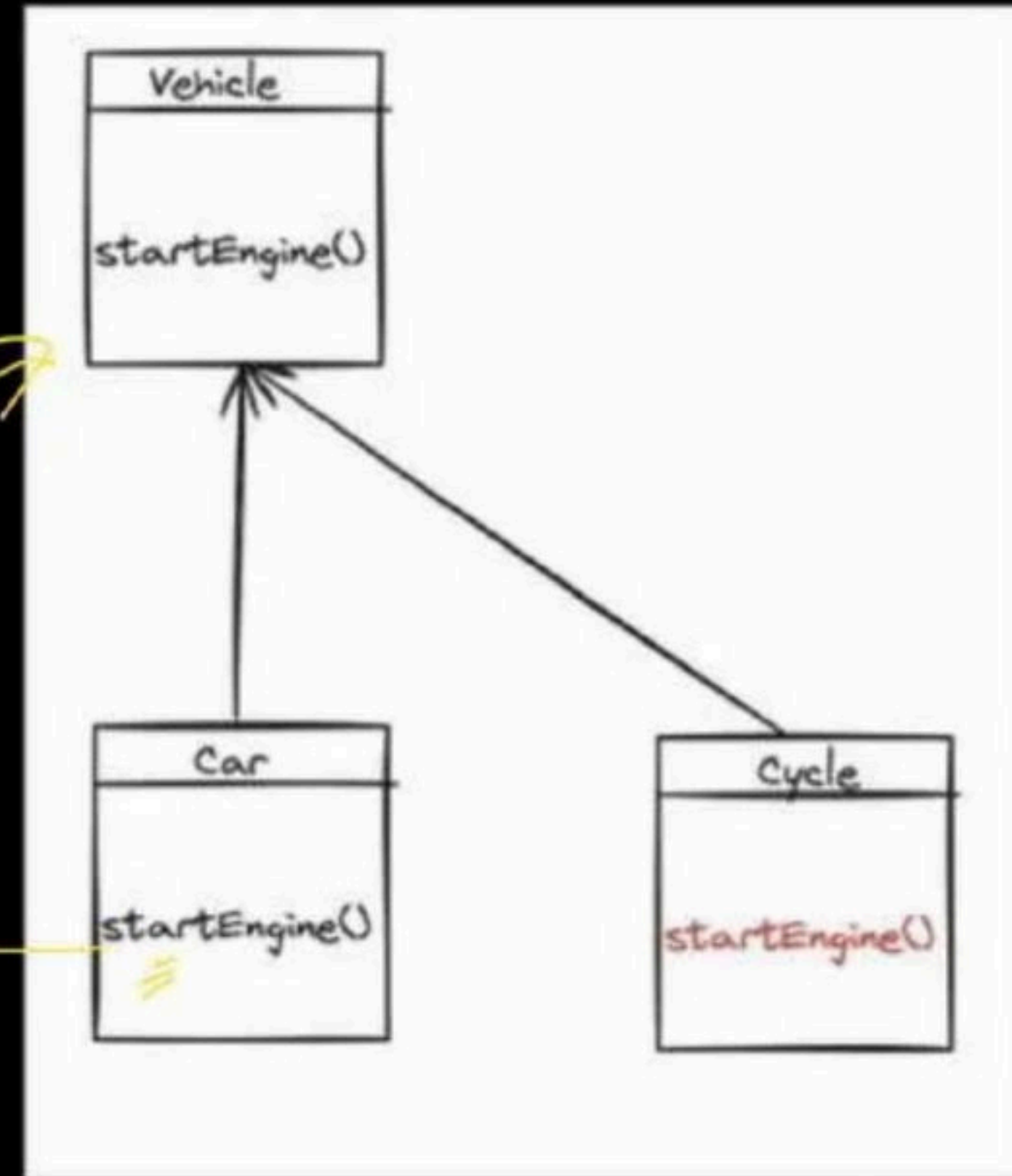
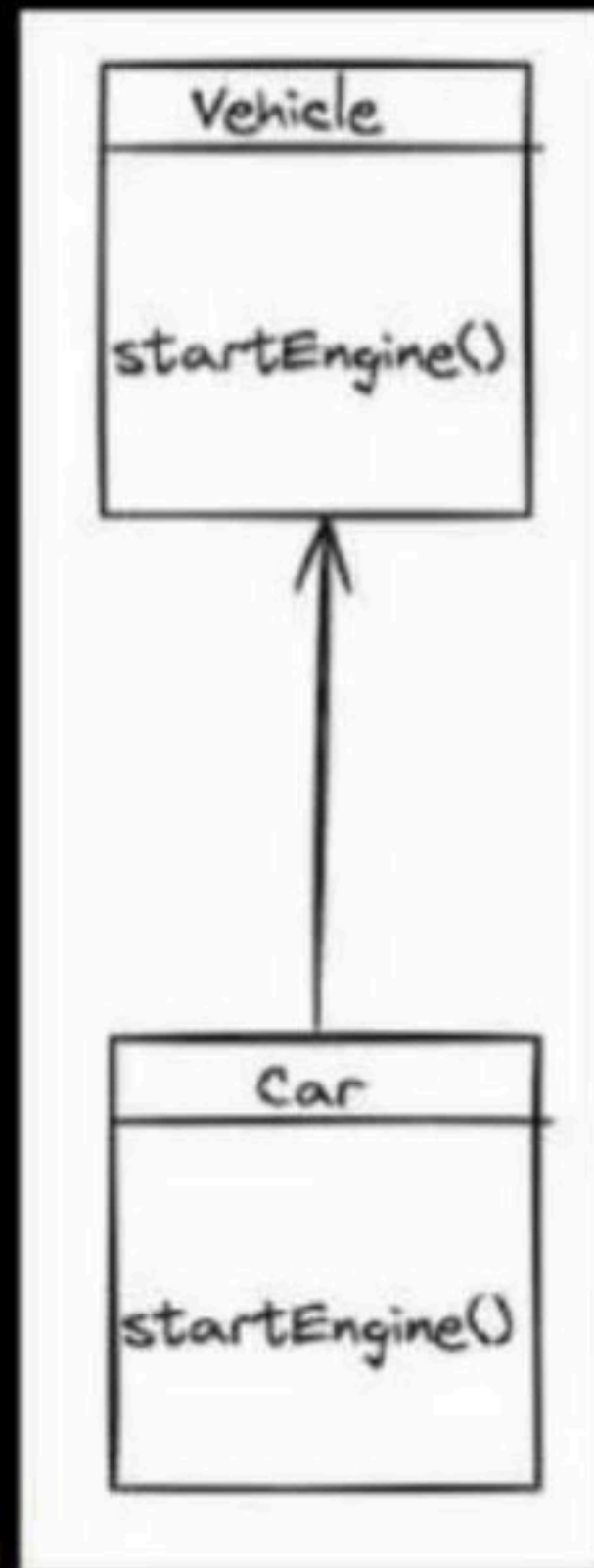
```
↓  
ISingBird {  
    sing();  
}
```

↳



Liskov Substitution Principle

1. Vehicle Example

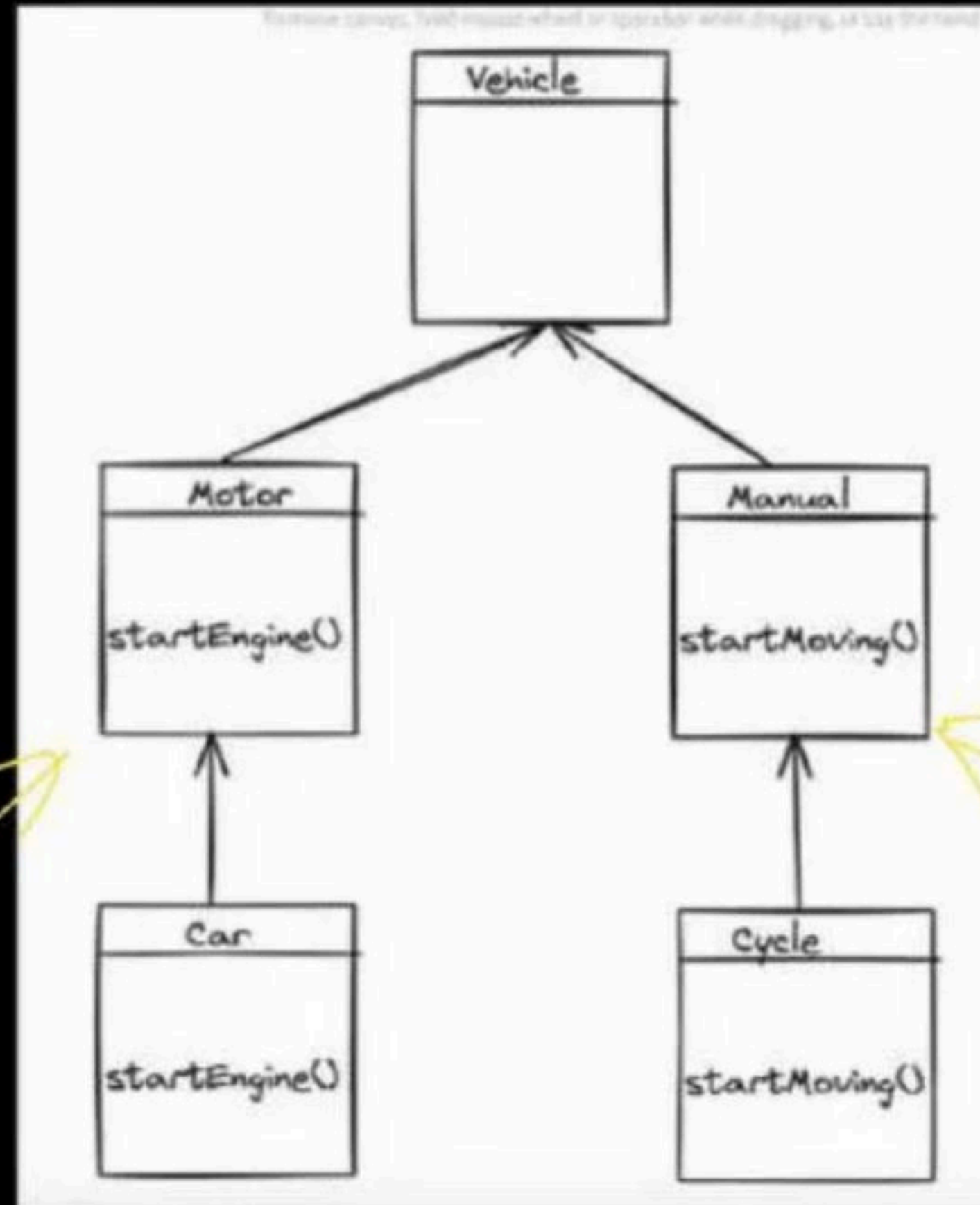


*fun(vehicle)
{
 v.startEngine();
}*

*>
fun(car)
fun(cycle)*

Liskov Substitution Principle

1. Vehicle Example



- Car is substitutable with its superclass, Motor, and Cycle is substitutable with its superclass, Manual, without breaking the functionality.
- Their methods can also override the methods of the superclass.

Truck

Rickshaw

Liskov Substitution Principle

1. Can a child go to work and make dinner?

Adult {

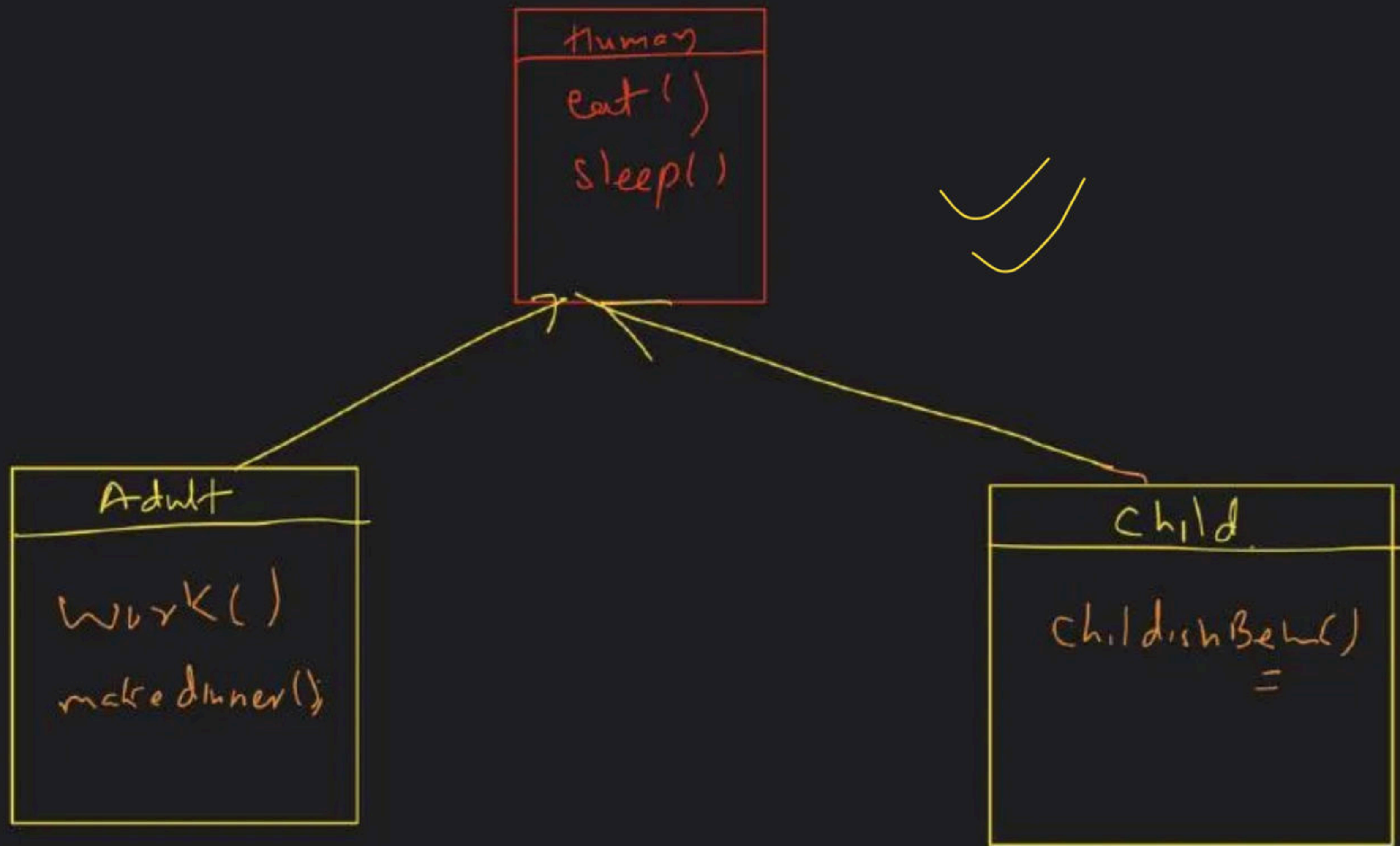
eat()
sleep()
work()
make dinner()
}

child {

eat()
sleep()
work()
make dinner();

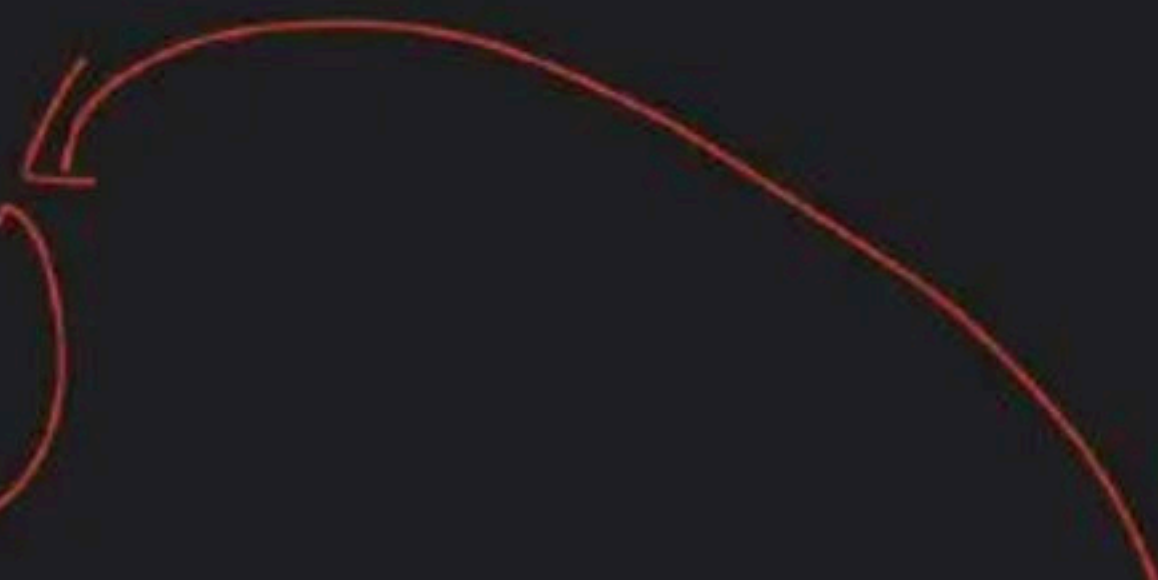
}

}



fun (Adult a)
- { a eat()
 a work() ✓
}

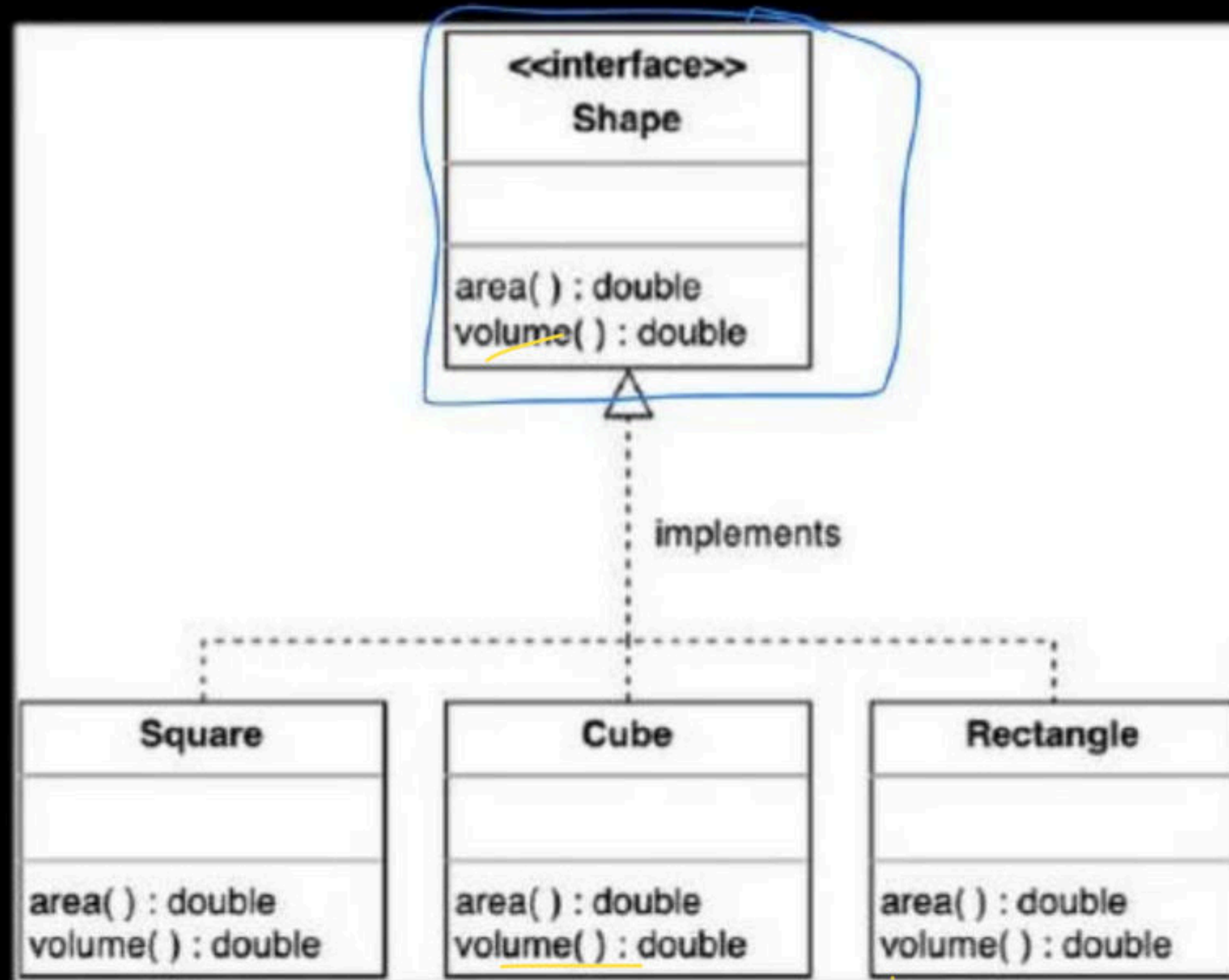
fun (c)
↓
child ref



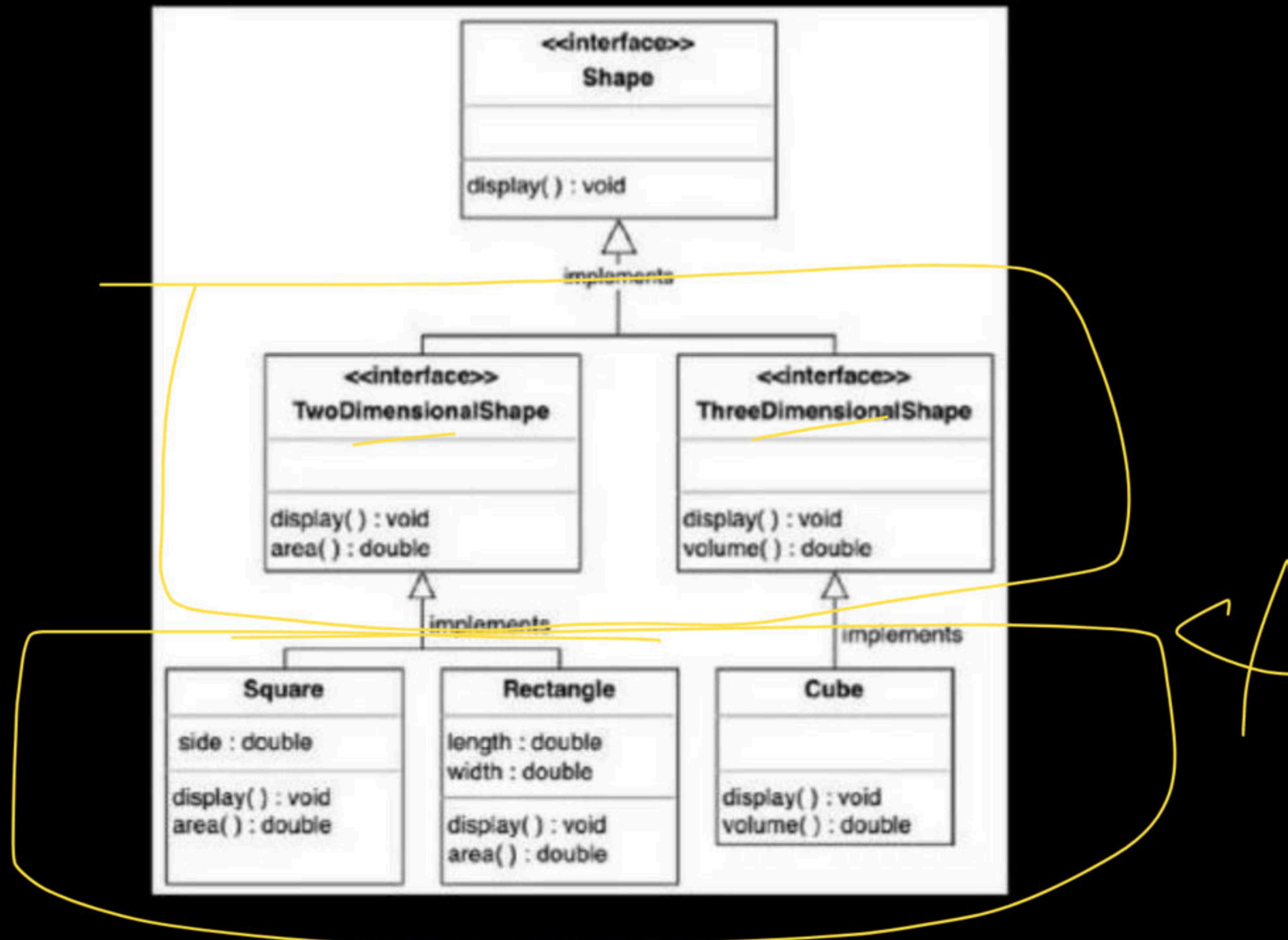
Interface Segregation Principle

1. The Interface Segregation Principle (ISP) is a design principle that does not recommend having methods that an interface would not use and require.
2. Therefore, it goes against having fat interfaces in classes and prefers having small interfaces with a group of methods, each serving a particular purpose.
3. To comply with the Interface Segregation Principle (ISP), it's important to design interfaces that are tailored to specific client needs instead of creating broad, all-purpose interfaces.
4. Do not build one pet interface make smaller and specific ones.

Interface Segregation Principle



Interface Segregation Principle



LSP $\Rightarrow$ asks you to make sure all child
classes have same behaviour as parent class.

ISP $\Rightarrow$ Create a diff interface for diff
responsibility, don't group unrelated
behaviour in one interface.

class Device {

void callBaba() {
 // father call.

}

}

(SP void

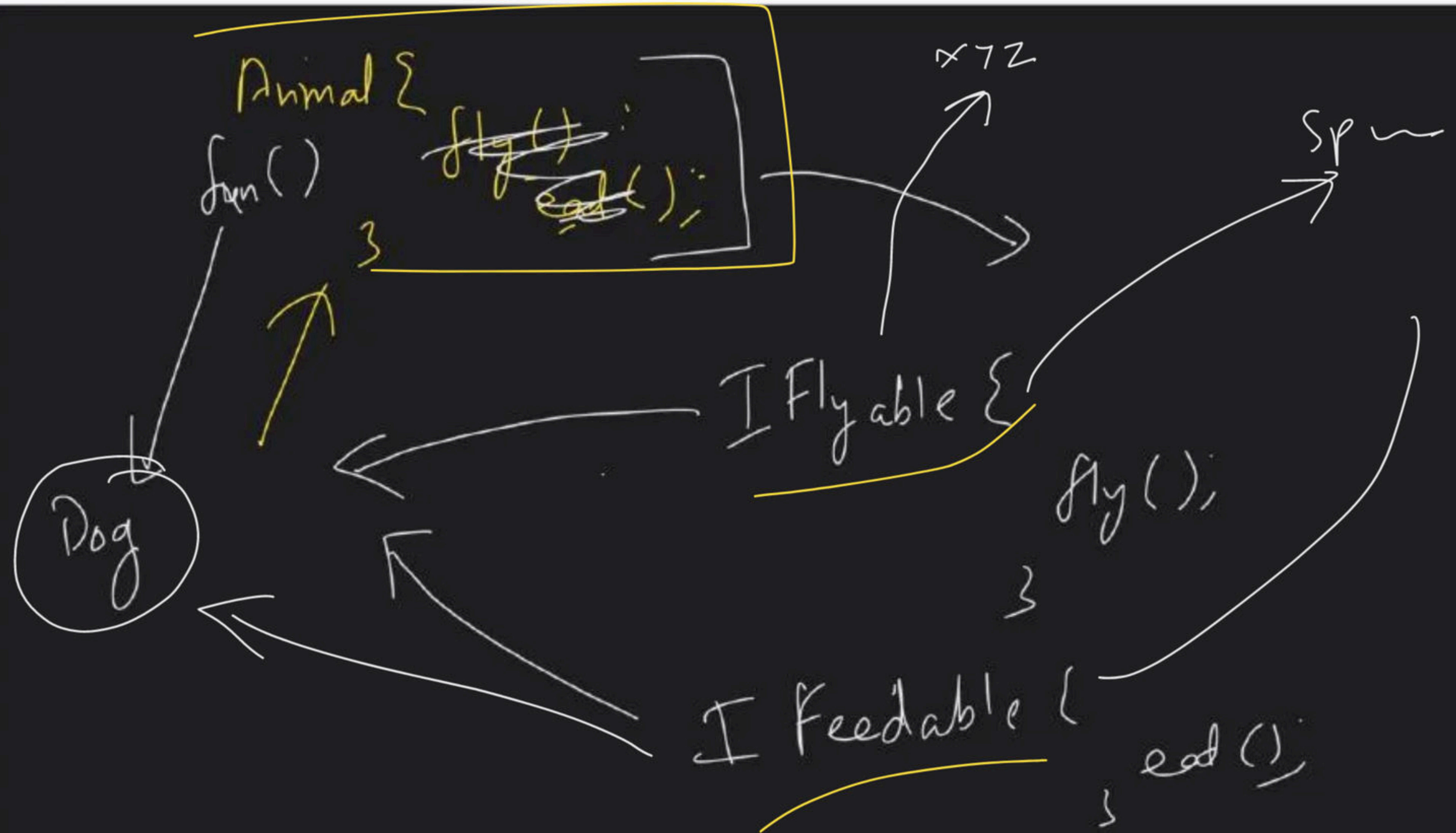
class Calc : Device {

void callBaba() {

 // exception, // can't do this

}

}

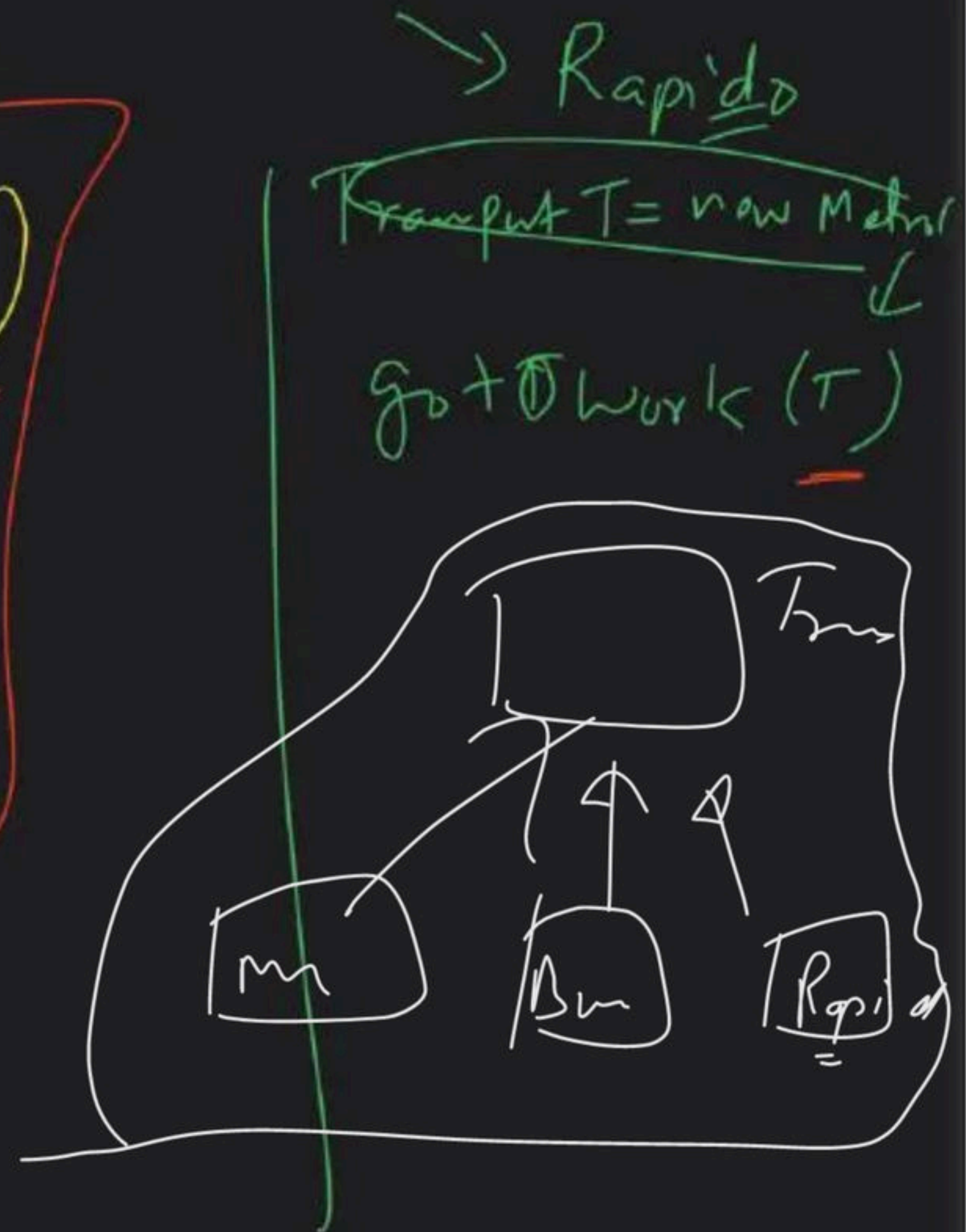
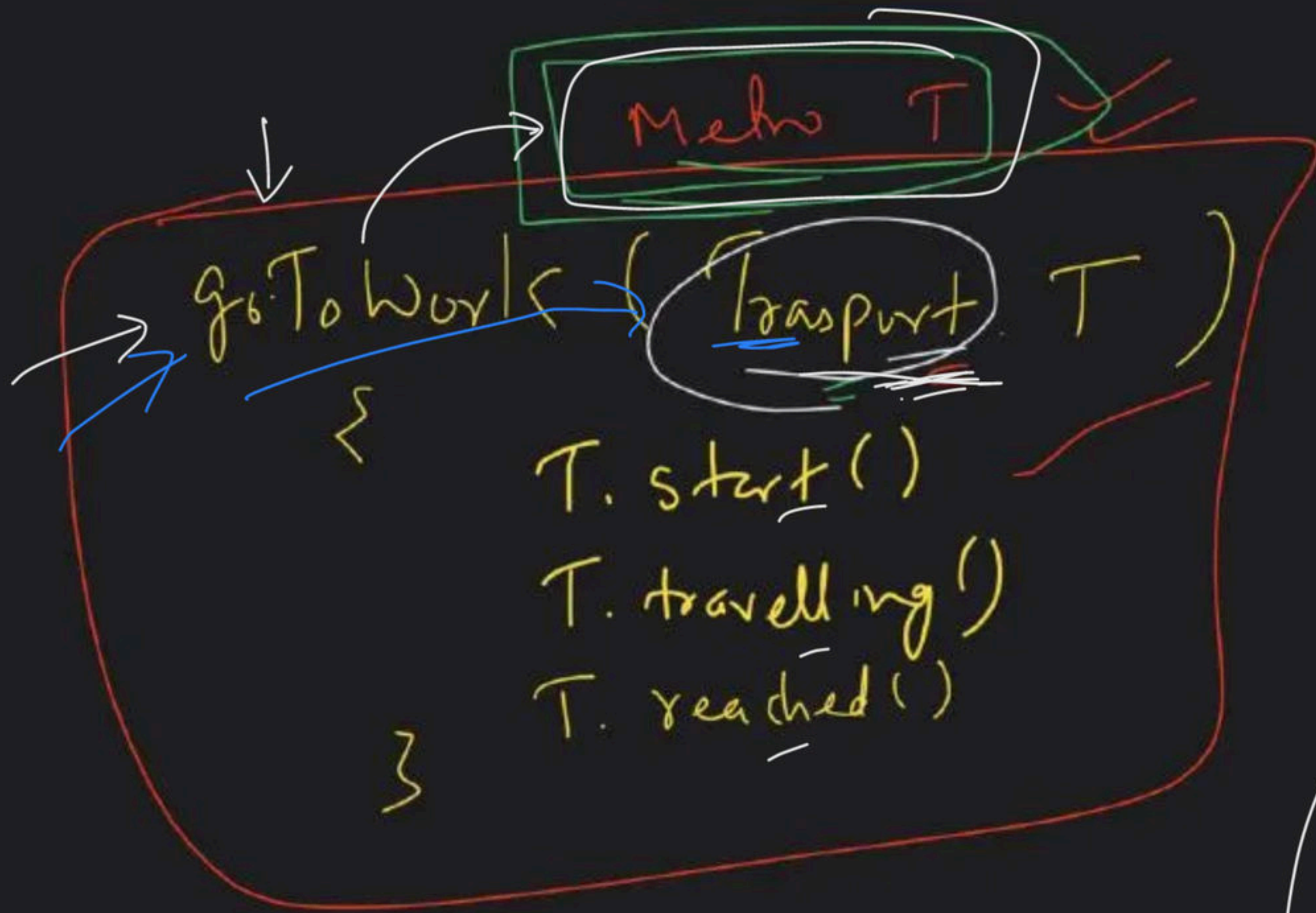


Dependency Inversion Principle

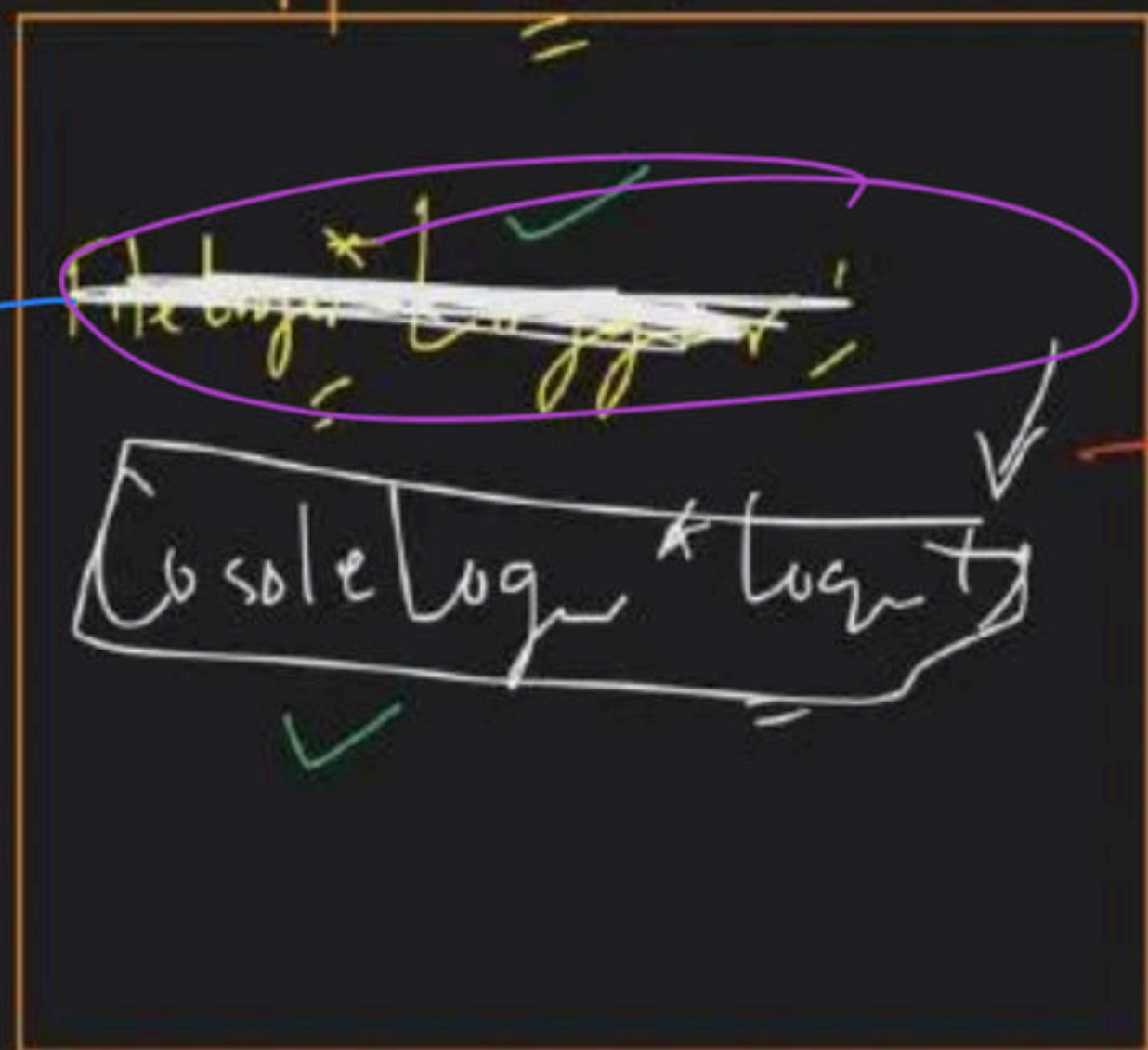
1. Never depend on everything concrete, only depend on Abstraction.
2. High level module should not depend on low level module. They should depend on Abstraction.
3. Able to change an implementation easily without altering the high level code.
4. By adhering to DIP, you can create systems that are resilient to change, as modifications to concrete implementations do not affect high-level modules.

✓
✓
fun Sparrow(Sparrow sp)
 {
 }
}

Bird
 ↓
fun (Base B)
 {
 B.fun
 }
 }



Application

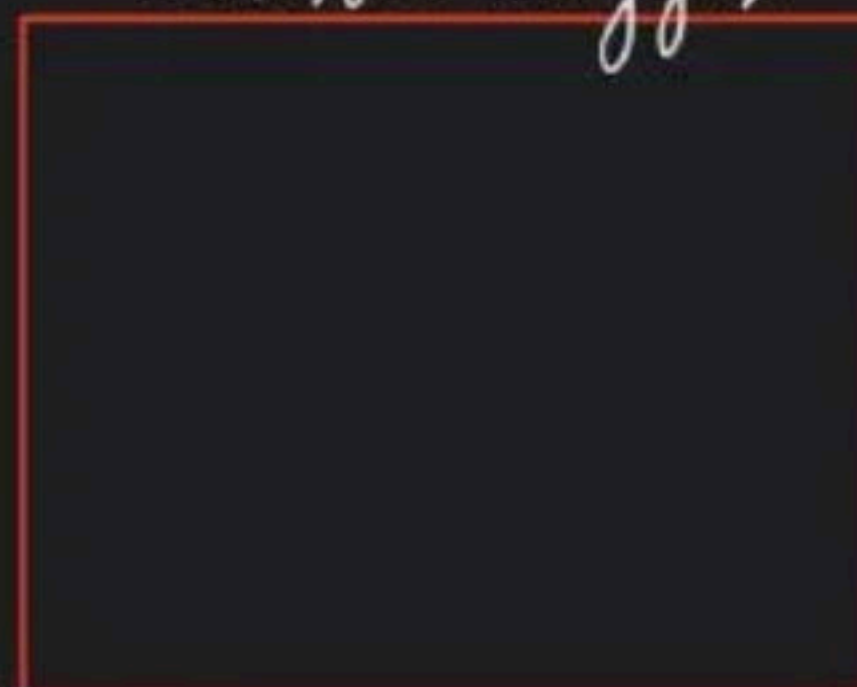


FileLogger



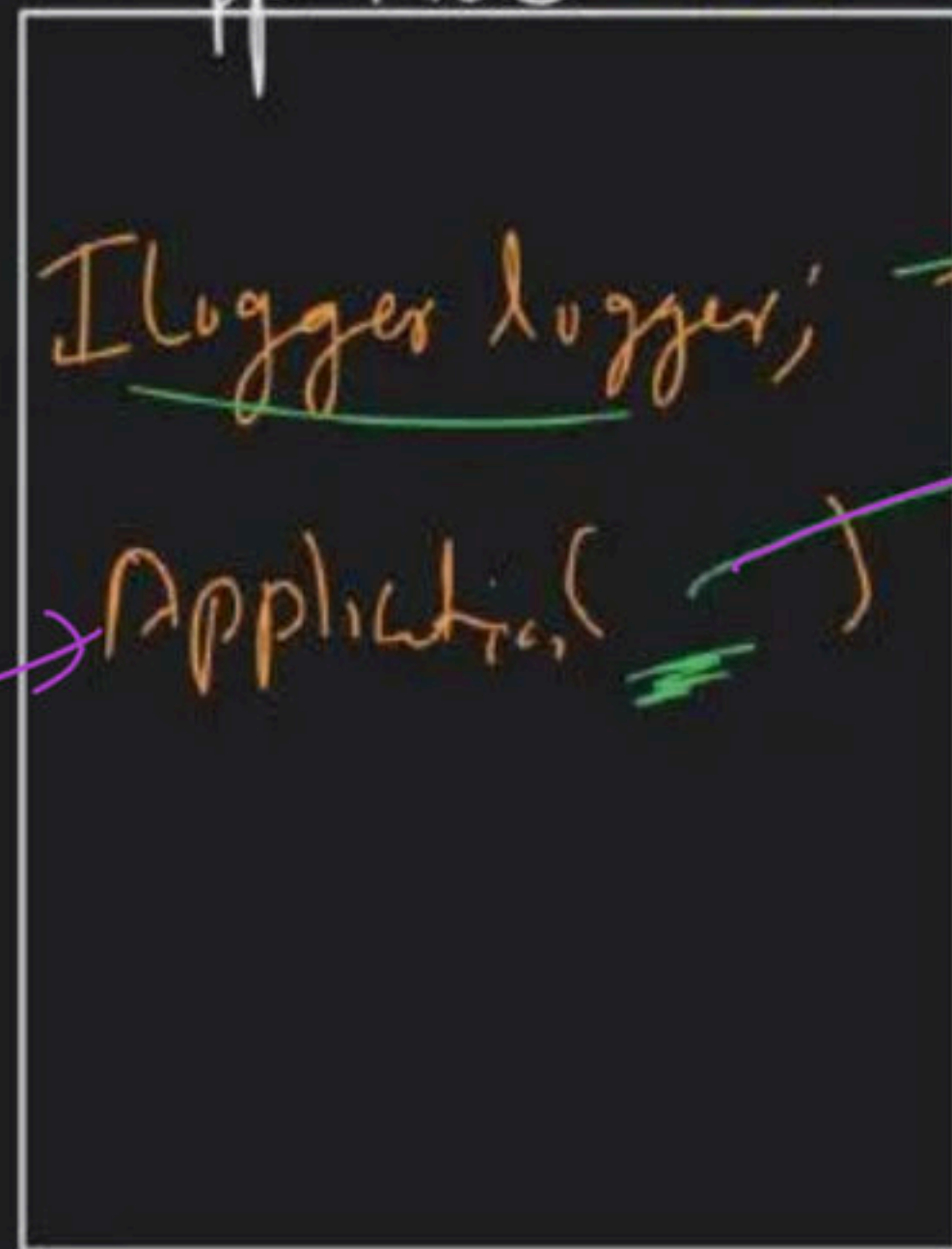
High Level
Module

Console Logger



Problem

Application



ctor

ILogger logger

① ILogger

Log();

ConsoleLogger

Log() {
}

FileLogger
Log() {
}

Application app = new Application(ConsoleLogger)

